Below is the **exact wiring code (1 → 5)** for **2.pdf templates**:

Templates covered:

1. ✅ Incident Resolution Summary → `incidentresolution`
2. ✅ Post-Incident Review (RCA) → `pirrca`
3. ✅ Deployment Runbook → `deploymentrunbook`
4. ✅ User Story Technical Design → `technicaldesign`

Use these mode IDs consistently.

✅ 1️⃣ Add Mode Detection in `loadJSON()`

Add inside your existing `loadJSON()` after parsing `data`:

```
// ---- 2.pdf Template Mode Detection ----

// Incident Resolution Summary
if (data.incident_number && data.resolution && data.impact?.impact_start) {
  switchMode('incidentresolution');
}

// PIR / RCA Report
else if (data.incident_number && data.executive_summary &&
data.root_cause_analysis) {
  switchMode('pirrca');
}

// Deployment Runbook
else if (data.runbook_title && data.execution_steps) {
  switchMode('deploymentrunbook');
}

// User Story Technical Design
else if (data.story_id && data.data_model_changes) {
  switchMode('technicaldesign');
}
```

✅ 2️⃣ Add Generation Case in `generate()`

Inside your `generate()` function:

```
// ---- 2.pdf Template Generators ----

else if (currentMode === 'incidentresolution') {
```

```
    result = await generateIncidentResolutionSummary(currentData);
  }
  else if (currentMode === 'pirrca') {
    result = await generatePostIncidentReviewRCA(currentData);
  }
  else if (currentMode === 'deploymentrunbook') {
    result = await generateDeploymentRunbook(currentData);
  }
  else if (currentMode === 'technicaldesign') {
    result = await generateUserStoryTechnicalDesign(currentData);
  }
```

✅ 3️⃣ Add Validation in `showValPanel()`

Add these blocks inside your validation logic.

✅ Incident Resolution Validation

```
else if (currentMode === 'incidentresolution') {

  tiles = [
    { l: 'Symptoms', n: (data.symptoms || []).length },
    { l: 'Resolution Steps', n: (data.resolution_steps || []).length },
    { l: 'Next Steps', n: (data.next_steps || []).length }
  ];

  if (!data.incident_number) {
    pip = 'warn';
    msg = 'Incident number missing';
  } else {
    msgs.push({ type: 'ok', text: 'Incident Resolution Summary ready' });
  }
}
```

✅ PIR / RCA Validation

```
else if (currentMode === 'pirrca') {

  tiles = [
    { l: 'Timeline Events', n: (data.timeline || []).length },
    { l: 'Corrective Actions', n: (data.corrective_actions || []).length },
    { l: 'Evidence Links', n: (data.evidence_links || []).length }
  ];
```

```
  if (!data.executive_summary) {
    pip = 'warn';
    msg = 'Executive summary missing';
  } else {
    msgs.push({ type: 'ok', text: 'PIR / RCA ready' });
  }
}
```

✅ Deployment Runbook Validation

```
else if (currentMode === 'deploymentrunbook') {

  tiles = [
    { l: 'Execution Steps', n: (data.execution_steps || []).length },
    { l: 'Rollback Steps', n: (data.rollback_steps || []).length },
    { l: 'Pre-Checks', n: (data.pre_checks || []).length }
  ];

  if (!data.runbook_title) {
    pip = 'warn';
    msg = 'Runbook title missing';
  } else {
    msgs.push({ type: 'ok', text: 'Deployment Runbook ready' });
  }
}
```

✅ Technical Design Validation

```
else if (currentMode === 'technicaldesign') {

  tiles = [
    { l: 'Data Model Changes', n: (data.data_model_changes || []).length },
    { l: 'Components', n: (data.implementation_components || []).length },
    { l: 'Risks', n: (data.risks || []).length }
  ];

  if (!data.story_id) {
    pip = 'warn';
    msg = 'Story ID missing';
  } else {
    msgs.push({ type: 'ok', text: 'Technical Design ready' });
  }
}
```

✅ 4️⃣ Add Preview Routing in `renderPreview()`

(File: `templateFunctions.js`)

Inside your existing `renderPreview()`:

```
// ---- 2.pdf Preview Routing ----

else if (currentMode === 'incidentresolution') {
  el.innerHTML = renderIncidentResolutionPreview(data);
}
else if (currentMode === 'pirrca') {
  el.innerHTML = renderPIRRCAPreview(data);
}
else if (currentMode === 'deploymentrunbook') {
  el.innerHTML = renderDeploymentRunbookPreview(data);
}
else if (currentMode === 'technicaldesign') {
  el.innerHTML = renderTechnicalDesignPreview(data);
}
```

✅ 5️⃣ Add Preview Functions

(Add AFTER your existing preview functions)

✅ Incident Resolution Preview

```
function renderIncidentResolutionPreview(data) {

  var html = '';
  html += '<div class="doc-cover">';
  html += '<div class="doc-cover-title">INCIDENT RESOLUTION SUMMARY</div>';
  html += '<div class="doc-cover-sub">' + escHtml(data.incident_number || '') +
'</div>';
  html += '</div>';

  if (data.short_description) {
    html += '<div class="doc-text"><strong>Description:</strong> ' +
      escHtml(data.short_description) + '</div>';
  }

  if (data.symptoms?.length) {
    html += '<div class="doc-section-title">Symptoms</div><ul>';
    data.symptoms.forEach(function(s){
```

```
      html += '<li>' + escHtml(s) + '</li>';
    });
    html += '</ul>';
  }

  if (data.resolution?.summary) {
    html += '<div class="doc-section-title">Resolution</div>';
    html += '<div class="doc-text">' + escHtml(data.resolution.summary) +
'</div>';
  }

  return html;
}
```

✅ PIR / RCA Preview

```
function renderPIRRCAPreview(data) {

  var html = '';
  html += '<div class="doc-cover">';
  html += '<div class="doc-cover-title">POST INCIDENT REVIEW (RCA)</div>';
  html += '<div class="doc-cover-sub">' + escHtml(data.incident_number || '') +
'</div>';
  html += '</div>';

  if (data.executive_summary) {
    html += '<div class="doc-section-title">Executive Summary</div>';
    html += '<div class="doc-text">' + escHtml(data.executive_summary) + '</div>';
  }

  if (data.timeline?.length) {
    html += '<div class="doc-section-title">Timeline</div>';
    html += '<table class="doc-table"><thead><tr><th>Time</th><th>Event</th>
<th>Owner</th></tr></thead><tbody>';

    data.timeline.forEach(function(t){
      html += '<tr><td>' + escHtml(t.time || '') + '</td>' +
              '<td>' + escHtml(t.event || '') + '</td>' +
              '<td>' + escHtml(t.owner || '') + '</td></tr>';
    });

    html += '</tbody></table>';
  }

  return html;
}
```

✅ Deployment Runbook Preview

```
function renderDeploymentRunbookPreview(data) {

  var html = '';
  html += '<div class="doc-cover">';
  html += '<div class="doc-cover-title">DEPLOYMENT RUNBOOK</div>';
  html += '<div class="doc-cover-sub">' + escHtml(data.runbook_title || '') +
'</div>';
  html += '</div>';

  if (data.execution_steps?.length) {
    html += '<div class="doc-section-title">Execution Steps</div>';
    html += '<ol>';

    data.execution_steps.forEach(function(s){
      html += '<li><strong>' + escHtml(s.action || '') + '</strong></li>';
    });

    html += '</ol>';
  }

  return html;
}
```

✅ Technical Design Preview

```
function renderTechnicalDesignPreview(data) {

  var html = '';
  html += '<div class="doc-cover">';
  html += '<div class="doc-cover-title">TECHNICAL DESIGN</div>';
  html += '<div class="doc-cover-sub">' + escHtml(data.story_id || '') + '</div>';
  html += '</div>';

  if (data.summary) {
    html += '<div class="doc-section-title">Summary</div>';
    html += '<div class="doc-text">' + escHtml(data.summary) + '</div>';
  }

  if (data.data_model_changes?.length) {
    html += '<div class="doc-section-title">Data Model Changes</div>';
    html += '<ul>';
    data.data_model_changes.forEach(function(d){
      html += '<li><strong>' + escHtml(d.table || '') + '</strong> - ' +
              escHtml(d.change || '') + '</li>';
```

```
      });
      html += '</ul>';
    }

    return html;
  }
```

✅ Done

You now have complete:

- ✅ Mode detection
- ✅ Generator routing
- ✅ Validation panel support
- ✅ Preview routing
- ✅ Preview UI rendering

for all **4 templates from 2.pdf** wired exactly like your existing system.

If you'd like, I can now generate the matching **index.html mode-tab buttons + gem-links entries** for these 4 templates so they appear cleanly in your UI grid.